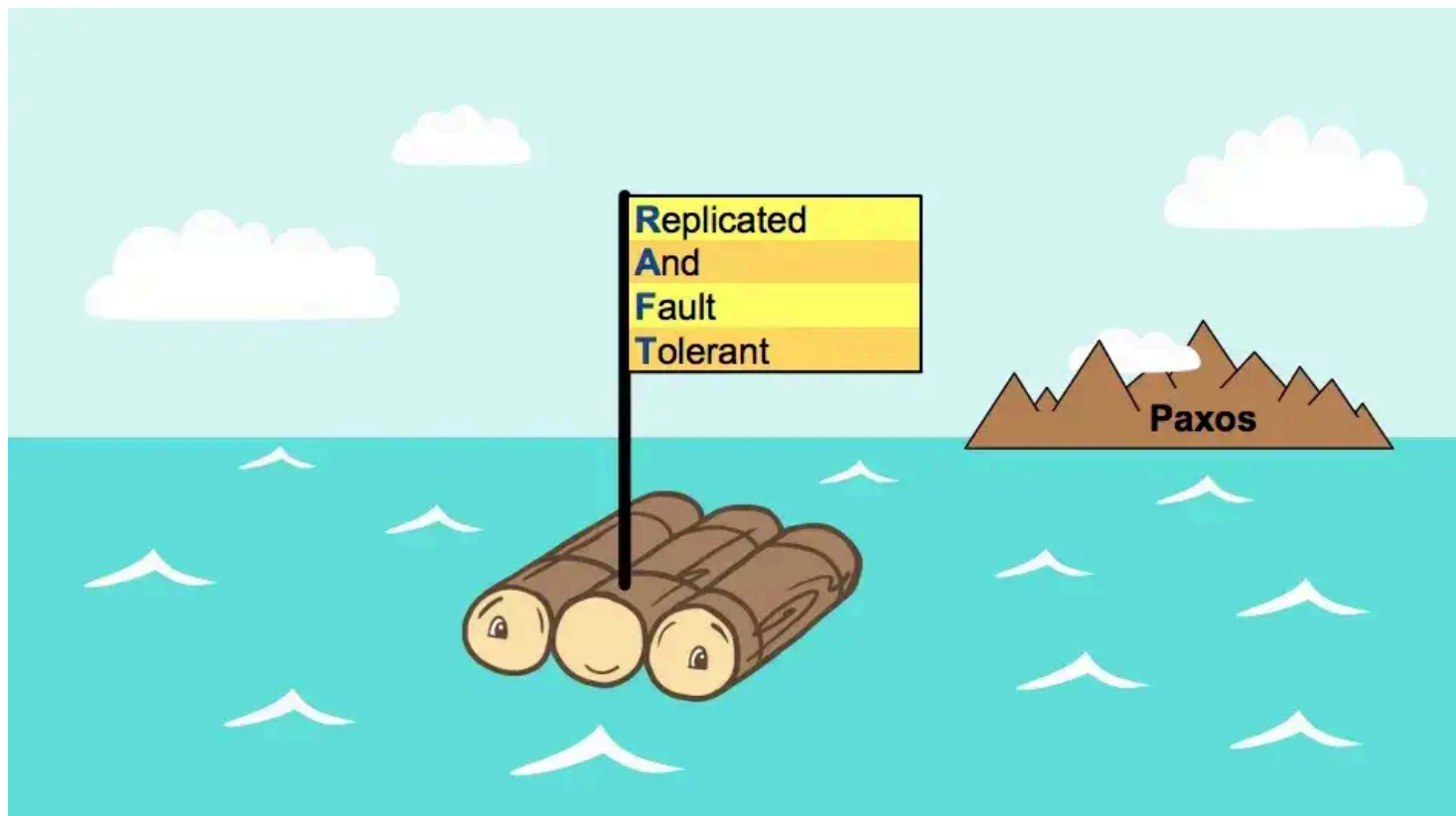




【译文】Raft协议：In Search of an Understandable Consensus Algorithm (Extended Version) 大名鼎鼎的分布式共识算法

GFS无法自动化解解决master单点故障的问题，且单机性能终究有限；而VM FT又太底层，是在操作系统层完成了指令的全复制。是否有一种在软件层面能够解决分布式系统可用性的算法？这便是大名鼎鼎的Raft协议，他简化了Paxos协议，使用强leader的方式保证了简洁性。MIT 6.824 课程中花费了大量的时间来讲解这篇论文 [In Search of an Understandable Consensus Algorithm \(Extended Version\) \(2014\)](#)，同时本人在读这篇文章的时候很多细节也是一知半解的，如果以后有幸从事分布式系统的架构与开发应该会重读这篇经典。本文由 [Hedon](#) 翻译。

摘要 (Abstract)

Raft 是用来管理复制日志 (replicated log) 的一致性协议。它跟 multi-Paxos 作用相同，效率也相当。但是它的组织结构跟 Paxos 不同，也是因为 Raft 更简单的架构使得它更容易被理解，并且更容易在实际工程中得以实现。

为了让 Raft 更容易被理解，Raft 将共识算法的关键性因素切分成几个部分，比如：

- leader election (领导者选举)

- log replication（日志复制）
- safety（安全性）

并且 Raft 实施了一种更强的共识性以便减少必须要考虑的状态（states）的数量。

用户研究表明，对于学生来说，Raft 相比于 Paxos 是更容易学习的。

Raft 还包括一个用于解决**变更集群成员问题**的新机制，它使用重写多数来保证安全性。

1. 介绍（Introduction）

共识算法允许多台机器作为一个集群协同工作，并且在其中的某几台机器出故障时集群仍然能正常工作。正因为如此，共识算法在建立可靠的大规模软件系统方面发挥了重要作用。在过去十年中，Paxos [15,16] 主导了关于共识算法的讨论：大多数共识性的实现都是基于 Paxos 或受其影响，Paxos 已经成为教授学生关于共识知识的主要工具。

比较遗憾的是，尽管很多人一直在努力尝试使 Paxos 更易懂，Paxos 还是太难理解了。此外，Paxos 的架构需要复杂的改变来支持实际系统。这导致的结果就是系统开发者和学生在学生和使用 Paxos 过程中都很挣扎。

在我们自己与 Paxos 斗争之后，我们开始着手寻找一个新的共识算法，希望可以为系统开发和教学提供更好的基础。我们的方法是不寻常的，因为我们的主要目标是可理解性：我们可以设计一个比 Paxos 更适合用于实际工程实现并且更易懂的共识算法吗？

在该算法的设计中，重要的不仅是如何让算法起作用，还要清晰地知道该算法为什么会起作用。

这项工作的结果是一个称为 Raft 的共识性算法。在设计 Raft 时，我们使用了特定的技术来提高它的可理解性，包括：

- 分解（Raft 分离出三个关键点：leader election、log replication、safety）
- 减少状态空间（相比于 Paxos，Raft 降低了不确定性的程度和服务器之间的不一致）

一项针对 2 所大学共 43 名学生的用户研究表明，Raft 比 Paxos 更容易理解：在学习两种算法后，其中 33 名学生能够更好地回答 Raft 的相关问题。

Raft 在许多方面类似于现有的公式算法（尤其是 Oki、Liskov 的 Viewstamped Replication [29,22]），但它有几个新特性：

- **Strong leader（强领导性）**：相比于其他算法，Raft 使用了更强的领导形式。比如，日志条目只能从 leader 流向 follower（集群中除 leader 外其他的服务器）。这在使 Raft 更易懂的同时简化了日志复制的管理流程。
- **Leader election（领导选举）**：Raft 使用随机计时器来进行领导选举。任何共识算法都需要心跳机制（heartbeats），Raft 只需要在这个基础上，添加少量机制，就可以简单快速地解决冲突。

- **Membership changes (成员变更)**：Raft 在更改集群中服务器集的机制中使用了一个 **联合共识 (joint consensus)** 的方法。在联合共识 (joint consensus) 下，在集群配置的转换过程中，新旧两种配置大多数是重叠的，这使得集群在配置更改期间可以继续正常运行。

我们认为 Raft 跟 Paxos 以及其他共识算法相比是更优的，这不仅体现在教学方面，还体现在工程实现方面。

- 它比其他算法更简单且更易于理解
- 它被描述得十分详细足以满足实际系统的需要
- 它有多个开源实现，并被多家公司使用
- 它的安全性已被正式规定和验证
- 它的效率与其他算法相当

本文剩余部分：

所在节	内容
第 2 节	复制状态机问题 (replicated state machine problem)
第 3 节	Paxos 的优缺点
第 4 节	实现 Raft 易理解性的措施
第 5-8 节	Raft 共识性算法详细阐述
第 9 节	评估 Raft
第 10 节	其他相关工作

2. 复制状态机 (Replicated state machines)

共识算法一般都是在[复制状态机](#) [37] 的背景下实现的。在这种方法下，一组服务器在的状态机计算相同状态的相同副本，即使某些服务器崩溃，它们也可以继续运行。

复制状态机是用来解决分布式系统中的各种容错问题。比如说，具有单个 leader 的大规模的系统，如 GFS [8]，HDFS [38] 和 RAMCloud [33]，他们通常都使用单独的复制状态机来管理 leader election 和保存 leader 崩溃后重新选举所需的配置信息。像 Chubby [2] 和 ZooKeeper [11] 都是复制状态机。

复制状态机通常都是使用日志复制 (log replication) 来实现。如图1：每个服务器都保存着一份拥有一系列命令的日志，然后服务器上的状态机会按顺序执行日志中的命令。每一份日志中命令相同并且顺序也相同，因此每个状态机可以处理相同的[命令序列](#)。所以状态机是可确定的，每个状态机都执行相同的状态和相同的输出序列。

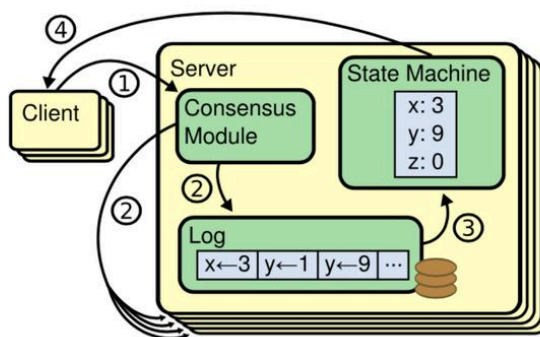


图1：复制状态机架构。共识算法管理一个包含来自客户端的状态机命令的复制日志。状态机处理来自日志的相同命令序列，因此它们会得到相同的输出。

共识算法的主要工作就是保证复制日志（replicated log）的一致性。每台服务器上的共识模块接收来自客户端的命令，并将这些命令添加到其日志当中。它（指共识模块）与其他服务器上的共识模块进行通信，以确保每台服务器上最终以相同的顺序包含相同的命令，即使部分服务器崩溃了，这个条件也可以满足。一旦命令被正确复制，每台服务器上的状态机就会按日志顺序处理它们，并将输出返回给客户端。这样就形成了高可用的复制状态机。

适用于实际系统的**共识算法通常都包含以下几点特征**：

- 它们确保在所有非拜占庭错误下的安全性，也就是从不返回一个错误的结果。（即使是网络延迟、分区、数据包丢失、数据包重复和数据包乱序）

拜占庭错误：

出现故障（crash 或 fail-stop，即不响应）但不会伪造信息的情况称为“非拜占庭错误”。伪造信息恶意响应的情况称为“拜占庭错误”，对应节点称为拜占庭节点。

- 只要任何大多数（过半）服务器是可运行的，并且可以互相通信和与客户端通信，那么共识算法就可用。假设服务器崩溃了，一小段时间后，它们很可能会根据已经稳定存储的状态来进行恢复，并重新加入集群。
- 它们在保证日志一致性上不依赖于时序：错误的时钟和极端消息延迟在最坏的情况下会产生影响可用性的一系列问题。
- 在通常情况下，只要集群中大部分（过半）服务器已经响应了单轮远程过程调用（RPC），命令就可以被视为完成。少数（一半以下）慢服务器不会影响整个系统的性能。

3. Paxos 存在的问题

在过去的十年间，Leslie Lamport 的 Paxos 协议 [15] 几乎成为共识性（consensus）的同义词。它是课堂上被教授最多的**共识协议**，大多数共识性的实现也是以它为起点。Paxos 首先定义了能在单个决策问题（例如单个复制日志条目）上达成共识的协议。我们将这个子集称为 single-decree Paxos。然后 Paxos 组合该协议的多个实例去实现一系列决策，比如日志（multi-Paxos）。Paxos 保证了安全性和活性，它也

支持改变集群中的成员，它的安全性也已经被论证了，并且大多数情况下都是高效的。

美中不足的是，Paxos 有两个严重的缺点：

1. Paxos 非常难理解

众所周知，Paxos 非常晦涩难懂，除非下了很大的功夫，很少有人能够成功理解它。因此，尽管目前已经有几个尝试希望将 Paxos [16,20,21] 解释得通俗易懂一些，而且这些解释都集中在 `single-decree Paxos`，但是它们还是很难懂。

在对 NSDI 2012 参会者的非正式调查中，我们发现很少人会喜欢 Paxos，即使是经验丰富的研究人员。我们自己也一直在跟 Paxos 作斗争，我们也无法完全理解整个 Paxos 协议，直到阅读了几个更简单的描述和自己设计了替代 Paxos 的协议，我们才对 Paxos 有了比较深刻的理解。但这个过程，花了将近一年。

我们推测 Paxos 这么晦涩难懂，主要是因为作者选择了 `Single-decree Paxos` 来作为基础。

`Single-decree Paxos` 非常搞人：它分为两个阶段，但是并没有对这两个阶段进行简单直观の説明，而且这两个阶段也不能分开了单独理解，所以使用者将就很难理解为什么该算法能起作用。

`Multi-Paxos` 的合成规则又增加了许多复杂性。我们相信，对多个决定（日志，并非单个日志条目）达成共识的总体问题可以用其他更直接和更明显的方式进行分解。

2. Paxos 没有为实际实现提供一个良好的基础

其中一个原因是没有广泛认同的针对 `Multi-Paxos` 的算法。Lamport 的描述主要是针对 `single-decree Paxos` 的，他描述了针对 `multi-Paxos` 的可能方法，但缺少了很多细节。

目前已经有人在尝试具体化和优化 Paxos，比如 [26]，[39] 和 [13]，但是这些尝试都互不相同并且它们跟 Lamport 描述的也不尽相同。虽然像 Chubby [4] 这样的系统已经实现了类 Paxos（Paxos-like）算法，但是他们并没有透露出很多的实现细节。

此外，Paxos 的架构对于构建实际系统来说其实是一个糟糕的设计，这是 `single-decree Paxos` 分解的另一个结果。举个例子，这对于独立选择地日志条目的集合，然后再将它们合并到顺序日志当中没有任何好处，这只会增加复杂性。围绕日志来设计系统是更加简单和高效的方法，其中新条目按受约束的顺序依次附加。另外一个问题是 Paxos 在其核心使用了**对称对等方法**（尽管它最终表明了这会被用作一种**性能优化**的弱领导模式）。这在只有一个决策的情况下是有意义的，但是尽管如此，还是很少有实际系统采用了这种方法。如果有一系列的决策需要制定，更简单和更快速的方法应该是首先选择一个 leader，然后由 leader 去协调这些决策。

因此，按照 Paxos 来实现的实际系统往往跟 Paxos 相差很大。几乎所有的实现都是从 Paxos 开始，然后在实现的过程中发现了一系列的难题，在解决难题的过程中，开发出了跟 Paxos 完全不一样的架构。这样既费时又容易出错，而且 Paxos 本身的晦涩难懂又使得问题变得更加严重。Paxos 公式可能是证明其正确性的一个很好的公式，但真正的实现与 Paxos 又相差很大，这证明了它其实没有什么价值。下面来自 Chubby 作者的评论非常典型：

在 Paxos 算法描述和现实实现系统之间有着巨大的鸿沟...（如果一直按照 Paxos 算法走下去），最终的系统往往会建立在一个还未被证明的协议之上[4]。

综合上述问题，我们觉得 Paxos 在教学端和系统构建端都没有提供一个良好的基础。考虑到共识性在大规模软件系统中的重要性，我们决定去尝试一下看看能不能设计一个替代 Paxos 并且具有更好特性的共识算法。Raft 就是这次实验的结果。

4. 为可理解性而设计

在设计 Raft 算法过程中我们有几个目标：

- 它必须为系统构建提供一个完整且实际的基础，这样才能大大减少开发者的工作
- 它必须在任何情况下都是安全的并且在典型的应用条件下是可用的，并且在正常情况下是高效的

但是我们最重要的目标，也是我们遇到的最大的挑战：

- 它必须具有易理解性，它必须保证能够被大多数人轻松地理解。而且它必须能够让人形成直观的认识，这样系统构建者才能在实现过程中对它进行不可避免的拓展。

在设计 Raft 算法的过程中，很多情况下我们需要在多个备选方案下做出抉择。在这种情况下，我们往往会基于可理解性来进行抉择：

- 解释各个备选方案的难度有多大？例如，它的状态空间有多复杂？它是否具有难以理解的含义？
- 对于一个读者来说，完成理解这个方案和方案中的各种含义是否简单？

我们意识到这一的分析具有高度的主观性。所以我们采取了两种通用的措施来解决这个问题。

1. 第一个措施就是众所周知的**问题分解**：只要有可能，我们就将问题划分成几个相对独立地解决、解释和理解的子问题。例如，Raft 算法被我们划分成 leader 选举、日志复制、安全性和成员变更几个部分。
2. 第二个措施是**通过减少状态的数量来简化状态空间**，尽可能地使系统变得更加连贯和尽可能地消除不确定性。很明显的一个例子就是，所有的日志都是不允许有空挡的，并且 Raft 限制了日志之间可能不一样的方式。尽管在大多数情况下我们都极力去消除不确定性，但是在某些情况下不确定性却可以提高可理解性。一个重要的例子就是随机化方法，它们虽然引入了不确定性，但是它们往往能够通过以类似的方式处理所有可能的选择来减少状态空间（随便选，没关系）。所有我们使用了随机化来简化 Raft 中的 leader election 算法。

5. Raft 共识算法（The Raft consensus algorithm）

Raft 是一种用来管理第 2 节中提到的复制日志（replicated log）的算法。图 2 是该算法的浓缩，可以作为参考。图 3 列举了该算法的一些关键特性。这两张图中的内容将会在后面

Raft 在实现共识算法的过程中，首先选举一个 distinguished leader，然后由该 leader 全权负责复制日志的一致性。Leader 从客户端接收日志条目，然后将这些日志条目复制给其他服务器，并且在保证安全性的情况下通知其他服务器将日志条目应用到他们的状态机中。拥有一个 leader 大大简化了对复制日志的管理流程。例如，leader 可以在不跟其他服务器商议的情况下决定新的日志条目应该存放在日志的什么位置，并且数据都是从 leader 流向其他服务器。当然了，一个 leader 可能会崩溃，也可能与其他服务器断开连接，那么这个时候，Raft 就会选举出一个新的 leader 出来。

通过选举一个 leader 的方式，Raft 将共识问题分解成三个独立的子问题，这些问题将会在接下来的子章节中进行讨论：

- **Leader election（领导选举）**
一个 leader 倒下之后，一定会有一个新的 leader 站起来。
- **Log replication（日志复制）**
leader 必须接收来自客户端的日志条目然后复制到集群中的其他节点，并且强制其他节点的日志和自己的保持一致。
- **Safety（安全性）**
Raft 中安全性的关键是图 3 中状态机的安全性：只要有任何服务器节点将一个特定的日志条目应用到它的状态机中，那么其他服务器节点就不能在同一个日志索引位置上存储另外一条不同的指令。第 5.4 节将会描述 Raft 如何保证这种特性，而且该解决方案在 5.2 节描述的选举机制上还增加了额外的限制。

在展示了 Raft 共识算法后，本章节将讨论可用性的一些问题以及时序在系统中的所用。

状态：

所有服务器上的持久性状态 (在响应 RPC 请求之前，已经更新到了稳定的存储设备)

参数	解释
currentTerm	服务器已知最新的任期（在服务器首次启动时初始化为0，单调递增）
votedFor	当前任期内收到选票的 candidateId，如果没有投给任何候选人 则为空
log[]	日志条目；每个条目包含了用于状态机的命令，以及领导人接收到该条目时的任期（初始索引为1）

所有服务器上的易失性状态

参数	解释
commitIndex	已知已提交的最高的日志条目的索引（初始值为0，单调递增）
lastApplied	已经被应用到状态机的最高的日志条目的索引（初始值为0，单调递增）

领导人（服务器）上的易失性状态 (选举后已经重新初始化)

参数	解释
nextIndex[]	对于每一台服务器，发送到该服务器的下一个日志条目的索引（初始值为领导人最后的日志条目的索引+1）
matchIndex[]	对于每一台服务器，已知的已经复制到该服务器的最高日志条目的索引（初始值为0，单调递增）

追加条目（AppendEntries）RPC：

由领导人调用，用于日志条目的复制，同时也被当做心跳使用

参数	解释
term	领导人的任期
leaderId	领导人 ID 因此跟随者可以对客户端进行重定向（译者注：跟随者根据领导人 ID 把客户端的请求重定向到领导人，比如有时客户端把请求发给了跟随者而不是领导人）
prevLogIndex	紧邻新日志条目之前的那个日志条目的索引
prevLogTerm	紧邻新日志条目之前的那个日志条目的任期
entries[]	需要被保存的日志条目（被当做心跳使用时，则日志条目内容为空；为了提高效率可能一次性发送多个）
leaderCommit	领导人的已知已提交的最高的日志条目的索引

返回值	解释
term	当前任期，对于领导人而言 它会更新自己的任期
success	如果跟随者所含有的条目和 prevLogIndex 以及 prevLogTerm 匹配上了，则为 true

接收者的实现：

1. 返回假 如果领导人的任期小于接收者的当前任期（译者注：这里的接收者是指跟随者或者候选人）（5.1 节）
2. 返回假 如果接收者日志中没有包含这样一个条目 即该条目的任期在 prevLogIndex 上能和 prevLogTerm 匹配上（译者注：在接收者日志中 如果能找到一个和 prevLogIndex 以及 prevLogTerm 一样的索引和任期的日志条目 则继续执行下面的步骤 否则返回假）（5.3 节）

3. 如果一个已经存在的条目和新条目（译者注：即刚刚接收到的日志条目）发生了冲突（因为索引相同，任期不同），那么就删除这个已经存在的条目以及它之后的所有条目（5.3 节）
4. 追加日志中尚未存在的任何新条目
5. 如果领导人的已知已提交的最高日志条目的索引大于接收者的已知已提交最高日志条目的索引（`leaderCommit > commitIndex`），则把接收者的已知已经提交的最高的日志条目的索引 `commitIndex` 重置为 领导人的已知已经提交的最高的日志条目的索引 `leaderCommit` 或者是 上一个新条目的索引 取两者的最小值

请求投票（RequestVote）RPC：

由候选人负责调用用来征集选票（5.2 节）

参数	解释
<code>term</code>	候选人的任期号
<code>candidateId</code>	请求选票的候选人的 ID
<code>lastLogIndex</code>	候选人的最后日志条目的索引值
<code>lastLogTerm</code>	候选人最后日志条目的任期号

返回值	解释
<code>term</code>	当前任期号，以便于候选人去更新自己的任期号
<code>voteGranted</code>	候选人赢得了此张选票时为真

接收者实现：

1. 如果 `term < currentTerm` 返回 `false`（5.2 节）
2. 如果 `votedFor` 为空或者为 `candidateId`，并且候选人的日志至少和自己一样新，那么就投票给他（5.2 节，5.4 节）

所有服务器需遵守的规则：

所有服务器：

- 如果 `commitIndex > lastApplied`，则 `lastApplied` 递增，并将 `log[lastApplied]` 应用到状态机中（5.3 节）
- 如果接收到的 RPC 请求或响应中，任期号 `T > currentTerm`，则令 `currentTerm = T`，并切换为跟随者状态（5.1 节）

跟随者（5.2 节）：

- 响应来自候选人和领导人的请求

- 如果在超过选举超时时间的情况之前没有收到**当前领导人**（即该领导人的任期需与这个跟随者的当前任期相同）的心跳/附加日志，或者是给某个候选人投了票，就自己变成候选人

候选人（5.2 节）：

- 在转变成候选人后就立即开始选举过程
 - 自增当前的任期号（currentTerm）
 - 给自己投票
 - 重置选举超时计时器
 - 发送请求投票的 RPC 给其他所有服务器
- 如果接收到大多数服务器的选票，那么就变成领导人
- 如果接收到来自新的领导人的附加日志（AppendEntries）RPC，则转变成跟随者
- 如果选举过程超时，则再次发起一轮选举

领导人：

- 一旦成为领导人：发送空的附加日志（AppendEntries）RPC（心跳）给其他所有的服务器；在一定的空余时间之后不停的重复发送，以防止跟随者超时（5.2 节）
- 如果接收到来自客户端的请求：附加条目到本地日志中，在条目被应用到状态机后响应客户端（5.3 节）
- 如果对于一个跟随者，最后日志条目的索引值大于等于 nextIndex（

`lastLogIndex ≥ nextIndex`

)，则发送从 nextIndex 开始的所有日志条目：

- 如果成功：更新相应跟随者的 nextIndex 和 matchIndex
- 如果因为日志不一致而失败，则 nextIndex 递减并重试

- 假设存在 N 满足 $N > \text{commitIndex}$, 使得大多数的 $\text{matchIndex}[i] \geq N$ 以及 $\log[N].\text{term} == \text{currentTerm}$ 成立, 则令 $\text{commitIndex} = N$ (5.3 和 5.4 节)

State

Persistent state on all servers:

(Updated on stable storage before responding to RPCs)

currentTerm	latest term server has seen (initialized to 0 on first boot, increases monotonically)
votedFor	candidateId that received vote in current term (or null if none)
log[]	log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex	index of highest log entry known to be committed (initialized to 0, increases monotonically)
lastApplied	index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:

(Reinitialized after election)

nextIndex[]	for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)
matchIndex[]	for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat; may send more than one for efficiency)
leaderCommit	leader's commitIndex

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

- Reply false if $\text{term} < \text{currentTerm}$ (§5.1)
- Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
- If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
- Append any new entries not already in the log
- If $\text{leaderCommit} > \text{commitIndex}$, set $\text{commitIndex} = \min(\text{leaderCommit}, \text{index of last new entry})$

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Receiver implementation:

- Reply false if $\text{term} < \text{currentTerm}$ (§5.1)
- If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

Rules for Servers

All Servers:

- If $\text{commitIndex} > \text{lastApplied}$: increment lastApplied, apply $\log[\text{lastApplied}]$ to state machine (§5.3)
- If RPC request or response contains term $T > \text{currentTerm}$: set $\text{currentTerm} = T$, convert to follower (§5.1)

Followers (§5.2):

- Respond to RPCs from candidates and leaders
- If election timeout elapses without receiving AppendEntries RPC from current leader or granting vote to candidate: convert to candidate

Candidates (§5.2):

- On conversion to candidate, start election:
 - Increment currentTerm
 - Vote for self
 - Reset election timer
 - Send RequestVote RPCs to all other servers
- If votes received from majority of servers: become leader
- If AppendEntries RPC received from new leader: convert to follower
- If election timeout elapses: start new election

Leaders:

- Upon election: send initial empty AppendEntries RPCs (heartbeat) to each server; repeat during idle periods to prevent election timeouts (§5.2)
- If command received from client: append entry to local log, respond after entry applied to state machine (§5.3)
- If last log index $\geq \text{nextIndex}$ for a follower: send AppendEntries RPC with log entries starting at nextIndex
 - If successful: update nextIndex and matchIndex for follower (§5.3)
 - If AppendEntries fails because of log inconsistency: decrement nextIndex and retry (§5.3)
- If there exists an N such that $N > \text{commitIndex}$, a majority of $\text{matchIndex}[i] \geq N$, and $\log[N].\text{term} == \text{currentTerm}$: set $\text{commitIndex} = N$ (§5.3, §5.4).

状态	
所有服务器上的持久状态: (在响应 RPC 之前在稳定存储上更新)	
当前期限	服务器看到的最新术语 (首次启动时初始化为 0, 单调增加)
投票成果	本任期内获得投票的候选人 ID (如果没有则为 null)
日志[]	日志条目; 每个条目包含状态机的命令, 以及领导者收到条目时的术语 (第一个索引为 1)
所有服务器上的不稳定状态:	
提交索引	已知已提交的最高日志条目的索引 (初始化为 0, 单调增加)
最后应用	应用于状态机的最高日志条目的索引 (初始化为 0, 单调增加)
领导者的不稳定状态: (选举后重新初始化)	
下一个索引[]	对于每个服务器, 发送到该服务器的下一个日志条目的索引 (初始化为领导者最后一个日志索引 + 1)
匹配索引[]	对于每个服务器, 已知在服务器上复制的最高日志条目的索引 (初始化为 0, 单调增加)

附加条目 RPC	
由领导者调用以复制日志条目 (§ 5.3); 也用作心跳 (§ 5.2)。	
论据:	
学期	领导任期
leaderId	这样关注者就可以重定向客户端
prevLogIndex	紧接在新日志条目之前的日志条目索引
prevLogTerm	prevLogIndex 条目的术语
条目[]	要存储的日志条目 (心跳时为空; 为了提高效率, 可能会发送多个)
领导者提交	领导者的提交索引
结果:	
术语	currentTerm, 如果跟随者包含与 prevLogIndex 和 prevLogTerm 匹配的条目, 则领导者将自身更新为 true
成功	
接收器实现:	
1. 如果 term < currentTerm 则回复 false (§ 5.1)	
2. 如果日志在 prevLogIndex 处不包含与 prevLogTerm 匹配的条目, 则回复 false (§ 5.3)	
3. 如果现有条目与新条目冲突 (索引相同但术语不同), 则删除现有条目及其后面的所有内容 (§ 5.3)	
4. 附加日志中尚未包含的任何新条目	
5. 如果 leaderCommit > commitIndex, 则设置 commitIndex = min(leaderCommit, 最后一个新条目的索引)	

请求投票 RPC	
候选人援引该条款来收集选票 (第 5.2 条)。	
论据:	
学期	候选人任期
候选人ID	候选人请求投票
最后的日志索引	候选人最后日志条目的索引 (§ 5.4)
最后一个日志术语	候选人最后一条日志条目的任期 (§ 5.4)
结果:	
学期	currentTerm, 对于候选人来说更新自己为 true 意味着候选人获得了选票
投票已批准	
接收器实现:	
1. 如果 term < currentTerm 则回复 false (§ 5.1)	
2. 如果 votedFor 为 null 或为 candidatesId, 且候选人的日志至少与接收者的日志一样最新, 则授予投票 (§ 5.2、§ 5.4)	

服务器规则	
所有服务器:	
• 如果 commitIndex > lastApplied: 增加 lastApplied, 将 log[lastApplied] 应用于状态机 (§ 5.3)	
• 如果 RPC 请求或响应包含术语 T > currentTerm: 设置 currentTerm = T, 转换为跟随者 (§ 5.1)	
关注者 (§ 5.2):	
• 回应候选人和领导人的 RPC	
• 如果选举超时, 且未收到当前领导者发来的 AppendEntries RPC 或未向候选人授予投票: 则转换为候选人	
候选人 (§ 5.2):	
• 转换为候选人后, 开始选举:	
• 增加当前期限	
• 为自己投票	
• 重置选举计时器	
• 向所有其他服务器发送请求投票 RPC	
• 如果收到大多数服务器的投票: 成为领导者	
• 如果从新的领导者收到 AppendEntries RPC: 转换为追随者	
• 如果选举超时: 开始新的选举	
领导:	
• 选举后: 向每个服务器发送初始空的 AppendEntries RPC (心跳); 在空闲期间重复, 以防止选举超时 (§ 5.2)	
• 如果从客户端收到命令: 将条目附加到本地日志, 在条目应用于状态机后进行响应 (§ 5.3)	
• 如果关注者的最后一个日志索引 > nextIndex: 发送 AppendEntries RPC, 日志条目从 nextIndex 开始	
• 如果成功: 更新关注者的 nextIndex 和 matchIndex (§ 5.3)	
• 如果由于日志不一致导致 AppendEntries 失败: 减少 nextIndex 并重试 (§ 5.3)	
• 如果存在 N 并且 N > commitIndex, 则 matchIndex[i] ≥ N 的大多数, 且 log[N].term == currentTerm: 设置 commitIndex = N (§ 5.3、§ 5.4)。	

图 2: 一个关于 Raft 一致性算法的浓缩总结 (不包括成员变换和日志压缩)。

Election Safety: at most one leader can be elected in a given term. §5.2

Leader Append-Only: a leader never overwrites or deletes entries in its log; it only appends new entries. §5.3

Log Matching: if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index. §5.3

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms. §5.4

State Machine Safety: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index. §5.4.3

Election Safety: 在一个特定的任期中至多选出来一个 leader (5.2)

Leader Append-Only: leader 从不重写或删除它自己的日志条目，它只新增条目 (5.3)

Log Matching: 如果两条日志包含相同的索引和任期的条目，那么该日志在从给定索引引起的所有条目中都是相同的。(5.3)

Leader Completeness: 如果一个日志条目在给定的任期中提交，那么该条目将出现在 leader 中的所有更高序号的任期中。(5.4)

State Machine Safety: 如果一个服务器将给定索引的日志条目应用到其状态机，那么其他服务器不会在同一索引上应用不同的日志条目。(5.4.3)

图 3: Raft 保证上述这些特性在任何时刻都是正确的。节号表示讨论每个属性的位置。

图 3: Raft 在任何时候都保证以上的各个特性。

5.1 Raft 基础 (Raft basics)

一个 Raft 集群中包含若干个服务器节点，5 是一个比较典型的数字，5 个服务器的集群可以容忍 2 个节点的失效。在任何时刻，集群中的每一个节点都只可能是以下是三种身份之一：

- **leader**：它会处理所有来自客户端的请求（如果一个客户端和 follower 通信，follower 会将请求重定向到 leader 上）
- **follower**：它们被动的：它们不会发送任何请求，只是简单的响应来自 leader 和 candidate 的请求
- **candidate**：这是用来选举一个新的 leader 的时候出现的一种临时状态，这将在第 5.2 节中详细描述

在正常情况下，集群中只有一个 leader，然后剩下的节点都是 follower。图 4 展示了这些状态和它们之间的转换关系，这些转换关系将会在接下来进行讨论。

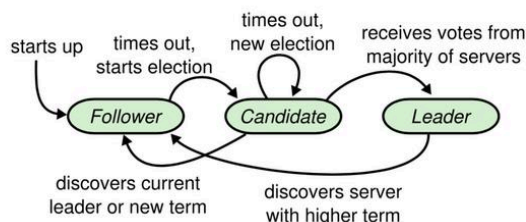


图 4: 服务器身份转换图

follower 只会响应来自其他服务器的请求。如果一个 follower 没有收到任何信息，它就会变成 candidate 并发起新一轮新的选举。如果一个 candidate 从全体节点中获得多数票（超过一半），那么它就会成为新的 leader。
leader 一般会一直工作直到它发生异常为止。

none -> follower: 集群刚启动，所有节点都是 follower

follower -> candidate: 集群中没有 leader 或 leader 挂了

candidate -> follower: candidate 没有得到过半的投票

candidate -> leader: candidate 得到过半的投票

candidate -> candidate: 没有选出 leader，重新新一轮选举

leader -> follower: leader 挂了，或者 leader 发现其他节点拥有比它更新的数据

如图 5 所示，Raft 将时间划分成任意长度的任期（term）。每一段任期从一次选举开始，在这个时候会有一个或者多个 candidate 尝试去成为 leader。如果某一个 candidate 赢得了选举，那么它就会在任期剩下的时间里承担一个 leader 的角色。在某些情况下，一次选举无法选出 leader，这个时候这个任期会以没有 leader 而结束。同时一个新的任期（包含一次新的选举）会很快重新开始。这是因为 Raft 会保证在

任意一个任期内，至多有一个 leader。

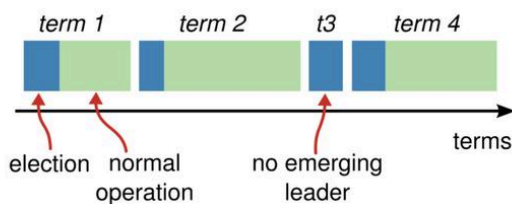


图 5：时间被分割为多个任期，每个任期都从一次选举开始。
在完成一次 leader election 之后，一个 leader 就会一直管理集群直到任期结束。
有时候选举也会失败，也就是任期结束时没有选出新的 leader。（t3）
在不同的服务器上，可以在不同的时间观察到不同的任期之间的转换。

集群中不同的服务器观察到的任期转换的次数也许是不同的，在某些情况下，一个节点可能没有观察到 leader 选举过程甚至是整个任期过程。

任期在 Raft 中还扮演着一个逻辑时钟（logical clock）的角色，这使得服务器可以发现一些过期的信息，比如过时的 leader。

每一个节点都存储着一个当前任期号（current term number），该任期号会随着时间单调递增。节点之间通信的时候会交换当前任期号，如果一个节点的当前任期号比其他节点小，那么它就将自己的任期号更新为较大的那个值。如果一个 candidate 或者 leader 发现自己的任期号过期了，它就会立刻回到 follower 状态。如果一个节点接收了一个带着过期的任期号的请求，那么它会拒绝这次请求。

Raft 算法中服务器节点之间采用 RPC 进行通信，一般的共识算法都只需要两种类型的 RPC。

- **RequestVote RPCs（请求投票）**：由 candidate 在选举过程中发出（5.2 节中描述）
- **AppendEntries RPCs（追加条目）**：由 leader 发出，用来做日志复制和提供心跳机制（5.3 节中描述）。

在第 7 节中为了在节点之间传输快照（snapshot）增加了第三种 RPC。当节点没有及时的收到 RPC 的响应时，会进行重试，而且节点之间都是以并行（parallel）的方式发送 RPC 请求，以此来获得最佳的性能。

5.2 Leader 选举（Leader election）

Raft 采用一种心跳机制来触发 leader 选举。当服务器启动的时候，他们都会称为 follower。一个服务器节点只要从 candidate 或者 leader 那接收到有效的 RPC 就一直保持 follower 的状态。Leader 会周期性地向所有的 follower 发起心跳来维持自己的 leader 地位，所谓心跳，就是不包含日志条目的 AppendEntries RPC。如果一个 follower 在一段时间内没有收到任何信息（这段时间我们称为**选举超时 election timeout**），那么它就会假定目前集群中没有一个可用的 leader，然后开启一次选举来选择一个新的 leader。

开始进行选举的时候，一个 follower 会自增当前任期号然后切换为 candidate 状态。然后它会给自己投票，同时以并行的方式发送一个 RequestVote RPCs 给集群中的其他服务器节点（企图得到它们的投票）。一个 candidate 会一直保持当前状态直到以下的三件事之一发生（这些情况都会在下方的章节里分别讨论）：

- 它赢得选举，成为了 leader
- 其他节点赢得了选择，那么它会变成 follower
- 一段时间之后没有任何节点在选举中胜出

当一个 candidate 获取集群中过半服务器节点针对同一任期的投票时，它就赢得了这次选举并成为新的 leader。对于同一个任期，每一个服务器节点会按照 **先来先服务原则 (first-come-first-served)** 只投给一个 candidate（在 5.4 节会在投票上增加额外的限制）。这种要求获得过半投票才能成为 leader 的规则确保了最多只有一个 candidate 赢得此次选举（图 3 中的选举安全性）。只要有一个 candidate 赢得选举，它就会成为 leader。然后它就会向集群中其他节点发送心跳消息来确定自己的地位并阻止新的选举。

一个 candidate 在等待其他节点给它投票的时候，它也有可能接收到另外一个自称为 leader 的节点给它发过来的 AppendEntries RPC。

- 如果这个 leader 的任期号（这个任期号会在这次 RPC 中携带着）不小于这个 candidate 的当前任期号，那么这个 candidate 就会觉得这个 leader 是合法的，然后将自己转变为 follower 状态。
- 如果这个 leader 的任期号小于这个 candidate 的当前任期号，那么这个 candidate 就会拒绝这次 RPC，然后继续保持 candidate 状态。

第三种可能的结果是 candidate 既没有赢得选举也没有输。可以设想一下这么一个情况。所有的 follower 同时变成 candidate，然后它们都将票投给自己，那这样就没有 candidate 能得到超过半数的投票了，投票无果。当这种情况发生的时候，每个 candidate 都会进行一次超时响应 (time out)，然后通过自增任期号来开启一轮新的选举，并启动另一轮的 RequestVote RPCs。然而，如果没有额外的措施，这种无结果的投票可能会无限重复下去。

为了解决上述问题，Raft 采用 **随机选举超时时间 (randomized election timeouts)** 来确保很少产生无结果的投票，并且就算发生了也能很快地解决。为了防止选票一开始就被瓜分，选举超时时间是从一个固定的区间（比如，150-300ms）中随机选择。这样可以把服务器分散开来以确保在大多数情况下会只有一个服务器率先结束超时，那么这个时候，它就可以赢得选举并在其他服务器结束超时之前发送心跳（译者注：乘虚而入，不讲武德）。

同样的机制也可以被用来解决选票被瓜分 (split votes) 的情况。每个 candidate 在开始一轮选举之前会重置一个随机选举超时时间，然后一直等待直到结束超时状态。这样减少了在一次投票无果后再一次投票无果的可能性。9.3 节展示了该方案能够快速地选出一个 leader。

选举的例子可以很好地展现可理解性是如何指导我们在多种备选设计方案中做出抉择的。在一开始，我们本打算使用一种等级系统 (rank system)：每一个 candidate 被赋予一个一次的等级 (rank)，如果一个 candidate 发现另外一个 candidate 有着更高的登记，那么它就会返回 follower 状态，这样可以使高等级的 candidate 更加容易地赢得下一轮选举。但是我们发现这种方法在可用性方面会有一些小问题：**如果等级较高的服务器崩溃了，那么等级较低的服务器可能需要进入超时状态，然后重新成为一个 candidate。如果这种操作出现得太快，那么它可能会重进程去开启一轮新的选举。**经过我们对该算法做出了多次的调整，我们最终还是认为随机重试的方法更加通俗易懂。

5.3 日志复制 (Log replication)

Leader 一旦被选举出来，它就要开始为客户端的请求提供服务了。每一个客户端请求都包含一条将被复制状态机执行的命令。leader 会以一个新条目的方式将该命令追加到自己的日志中，并且以**同步**的方式向集群中的其他节点发起 AppendEntries RPCs，让它们复制该条目。当条目被安全地复制（何为安全复制，后面会介绍）之后，leader 会将该条目应用到自己的状态机中，状态机执行该指令，然后把执行的结果返回给客户端。如果 follower 崩溃了或者运行缓慢，或者网络丢包，leader 会不断地重试 AppendEntries RPCs（即使已经对客户端作出了响应）直到所有的 follower 都成功存储了所有的日志条目。

日志以图 6 展示的方式组织着。每条日志条目都存储着一条状态机指令和 leader 收到该指定时的任期号。日志条目中的任期号可以用来检测多个日志副本之间是否不一致，以此来保证图 3 中的某些性质。每个日志条目还有一个整数索引值来表明它在日志中的位置。

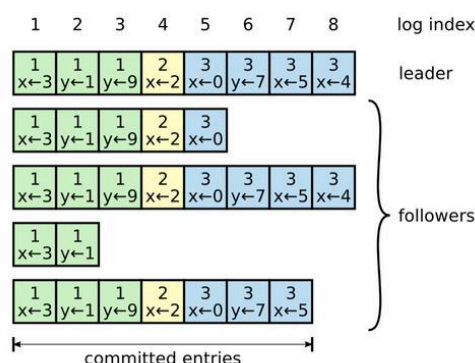


图6: 日志条目组织方式

日志由条目组成，条目按顺序编号。

每个条目都包含它在被创建时的所在的任期号，和状态机的命令。

如果某个条目应用于状态机是安全的，则认为该条目已经被提交。

那么问题就来了，**leader 什么时候会觉得把日志条目应用到状态机是安全的呢？**这种日志条目被称为已提交的日志条目。Raft 保证这种已提交的日志条目都是**持久化**的并且最终都会被所有可用的状态机执行。**一旦创建该日志条目的 leader 将它复制到过半的节点上时（比如图 6 中的条目 7），该日志条目就会被提交。**同时，leader 日志中该日志条目之前的所有日志条目也都会被提交，包括由之前的其他 leader 创建的日志条目。5.4 节会讨论在 leader 变更之后应用该规则的一些细节，并证明这种提交的规则是安全的。leader 会追踪它所知道的要提交的最高索引，并将该索引包含在未来的 AppendEntries RPC 中（包括心跳），以便其他的节点可以发现这个索引。一旦一个 follower 知道了一个日志条目被提交了。它就会将该日志条目按日志顺序应用到自己的状态机中。

我们设计 Raft 日志机制来使得不同节点上的日志之间可以保持高水平的一致性。这么做不仅简化了系统的行为也使得系统更加可预测，同时该机制也是保证安全性的重要组成部分。Raft 会一直维护着以下的特性，这些特性也同时构成了图 3 中的**日志匹配特性 (Log Matching Property)**：

- 如果不同日志中的两个条目有着相同的索引和任期值，那么它们就存储着相同的命令
- 如果不同日志中的两个条目有着相同的索引和任期值，那么他们之前的所有日志条目也都相同

第一条特性源于这样一个事实，在给定的一个任期值和给定的一个日志索引中，一个 leader 最多创建一个日志条目，而且日志条目永远不会改变它们在日志中的位置。

第二条特性是由 AppendEntries RPC 执行的一个**简单的一致性检查**所保证的。当 leader 发送一个 AppendEntries RPC 的时候，leader 会将前一个日志条目的索引位置和任期号包含在里面（紧邻最新的日志条目）。如果一个 follower 在它的日志中找不到包含相同索引位置和任期号的条目，那么它就会拒绝该新的日志条目。一致性检查就像一个**归纳步骤**：一开始空的日志状态肯定是满足日志匹配特性（Log Matching Property）的，然后一致性检查保证了日志扩展时的日志匹配特性。因此，当 AppendEntries RPC 返回成功时，leader 就知道 follower 的日志一定和自己相同（从第一个日志条目到最新条目）。

正常操作期间，leader 和 follower 的日志都是保持一致的，所以 AppendEntries 的一致性检查从来不会失败。但是，**如果 leader 崩溃了，那么就有可能造成日志处于不一致的状态**，比如说老的 leader 可能还没有完全复制它日志中的所有条目，它就崩溃了。这些不一致的情况会在一系列的 leader 和 follower 崩溃的情况下加剧。图 7 解释了什么情况下 follower 的日志可能和新的 leader 的日志不同。follower 可能会确实一些在新 leader 中有的日志条目，也有可能拥有一些新的 leader 没有的日志条目，或者同时存在。缺失或多出日志条目的情况有可能会涉及到多个任期。

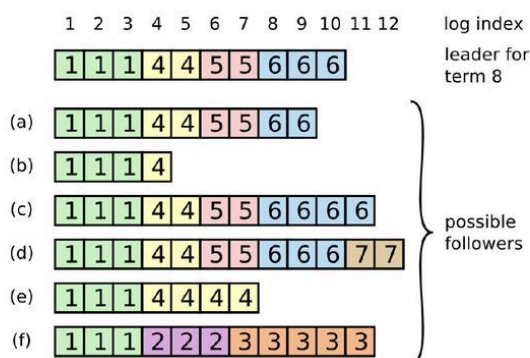


图7：leader 与 follower 日志不一致的情况

当一个 leader 成功当选时（最上面那条日志），follower 可能是以下 (a-f) 几种情况。每一个盒子表示一条日志条目，盒子内部的数字表示任期号。一个 follower 有可能会缺失一些日志条目 (a-b)，一个 follower 也有可能有一些未被提交的日志条目 (c-d)，或者两种情况都有 (e-f)。

例如，场景 f 有可能是这样发生的：f 对应的服务器在任期 2 的时候是 leader，它追加了一些日志条目到自己的日志中，一条日志还没提交就宕机了，但是它很快就恢复重启了，然后再在任期 3 重新被选举为 leader，又追加了一些日志条目到自己的日志中，在这些任期 2 和任期 3 的日志还没有被提交之前，该服务器又宕机了，并且在接下来的几个任期里一直处于宕机状态。

在 Raft 算法中，leader 通过强制 follower 复制 leader 日志来解决日志不一致的问题。也就是说，follower 中跟 leader 冲突的日志条目会被 leader 的日志条目所覆盖。5.4 节会证明通过增加一个限制，这种方式就可以保证安全性。

为了使 follower 的日志跟自己（leader）一致，leader 必须找到两者达成一致的最大的日志条目索引，删除 follower 日志中从那个索引之后的所有日志条目，并且将自己那个索引之后的所有日志条目发送给 follower。所有的这些操作都发生在 AppendEntries RPCs 的一致性检查的回复中。leader 维护着一个针对每一个 follower 的 **nextIndex**，这个 nextIndex 代表的就是 leader 要发送给 follower 的下一个日志条目的索引。**当选出一个新的 leader 时，该 leader 将所有的 nextIndex 的值都初始化为自己最后一个日志条目的 index 加 1（图 7 中的 11）。**如果一个 follower 的日志跟 leader 的是不一致的，那么下一次的 AppendEntries RPC 的一致性检查就会失败。**AppendEntries RPC 在被 follower 拒绝之后，**

leader 对 nextIndex 进行减 1，然后重试 AppendEntries RPC。最终 nextIndex 会在某个位置满足 leader 和 follower 在该位置及之前的日志是一致的，此时，AppendEntries RPC 就会成功，将 follower 跟 leader 冲突的日志条目全部删除然后追加 leader 中的日志条目（需要的话）。一旦 AppendEntries RPC 成功，follower 的日志就和 leader 的一致了，并且在该任期接下来的时间里都保持一致。

如果需要的话，下面的协议可以用来优化被拒绝的 AppendEntries RPCs 的个数。

比如说，当拒绝一个 AppendEntries RPC 的时候，follower 可以包含冲突条目的任期号和自己存储的那个任期的第一个 index。借助这些信息，leader 可以跳过那个任期内所有的日志条目来减少 nextIndex。这样就变成了每个有冲突日志条目的任期只需要一个 AppendEntries RPC，而不是每一个日志条目都需要一次 AppendEntries RPC。

在实践中，我们认为这种优化是没有必要的，因为失败不经常发生并且也不可能有很多不一致的日志条目。

通过上述机制，leader 在当权之后就不需要任何特殊的操作来使日志恢复到一致状态。leader 只需进行正常的操作，然后日志就能在回复 AppendEntries RPC 一致性检查的时候自动趋于一致。

上述这种日志复制机制展现了第 2 节中描述的 Raft 算法的共识特性：只要过半的节点能正常运行，Raft 就能接受、复制并处理新的日志条目。在通常情况下，一个新的条目可以在一轮 RPC 中被复制给集群中过半的节点，并且单个运行缓慢的 follower 并不会影响整个集群的性能。

译者注：总结

Leader 收到 Client 的写请求，向所有 Follower 发起一个日志同步请求，得到集群内过半节点（包括 Leader 自己）的响应，就推进 commitIndex，然后 apply 日志到状态机，再推进 applyIndex，返回 Client 成功。

状态机同步分为两轮 RPC 广播：

- 第一轮：同步日志 AppendEntries，得到过半节点回复，Leader 状态机推进，返回 Client 成功。
- 第二轮：在下一次的 AppendEntries 中附带上一次的 commitIndex，Follower 收到后，apply 日志条目到各自的状态机。（以应对 log 不同步的情况）

5.4 安全 (Safety)

前面的章节描述了 Raft 如何做 Leader Election 和 Log Replication。然而，到目前为止所讨论的机制并不能充分地保证每一个状态机会按相同的顺序执行相同的指令。比如说，一个 follower 可能会进入不可用状态，在此期间，leader 可能提交了若干的日志条目，**然后这个 follower 可能被选举为新的 leader 并且用新的日志条目去覆盖这些日志条目。**这样就会造成不同的状态机执行不同的指令的情况。

本节通过对 Leader Election 增加一个限制来完善 Raft 算法。这个限制保证了对于给定的任意任期号，该任期号对应的 leader 都包含了之前各个任期所有被提交的日志条目（图3 中的 Leader Completeness 性质）。有了这个限制，我们也可以使日志提交规则更加清晰。最后，我们会展示对于 Leader Completeness 性质的简要证明并且说该性质是如何保证状态机执行正确的行为的。

5.4.1 选举限制 (Election restriction)

在任何基于 leader 的共识算法中，leader 最终都必须存储所有已经提交的日志条目。在某些共识算法中，例如 Viewstamped Replication [22]，即使一个节点它一开始并没有包含所有已经提交的日志条目，它也有可能被选举为 leader。这些算法包含一些额外的机制来识别丢失的日志条目并将它们传送给新的 leader，这个机制要么发生在选举阶段，要么在选举完成之后很快进行。比较遗憾的是，这种方法会增加许多额外的机制，使得算法复杂性大大增加。Raft 使用了一种更加简单的方法，它可以保证新 leader 在当选时就包含了之前所有任期中已经提交的日志条目，根本就不需要再传送这些日志条目给新的 leader。这就意味着**日志条目的传送只有一个方向，那就是从 leader 到 follower，leader 从来不会覆盖本地日志中已有的日志。**

Raft 采用投票的方式来保证一个 candidate 只有拥有之前所有任期中已经提交的日志条目之后，才有可能赢得选举。一个 candidate 如果想要被选为 leader，那它就必须跟集群中超过半数的节点进行通信，这就意味这些节点中至少一个包含了所有已经提交的日志条目。**如果 candidate 的日志至少跟过半的服务器节点一样新，那么它就一定包含了所有以及提交的日志条目**，一旦有投票者自己的日志比 candidate 的还新，那么这个投票者就会拒绝该投票，该 candidate 也就不会赢得选举。

所谓“新”：

Raft 通过比较两份日志中的最后一条日志条目的索引和任期号来定义谁的日志更新。

- 如果两份日志最后条目的任期号不同，那么任期号大的日志更新
- 如果两份日志最后条目的任期号相同，那么谁的日志更长，谁就更新

5.4.2 提交之前任期内的日志条目 (Committing entries from previous terms)

译者注：注意！这一节「提交之前任期内的日志条目」这种操作 Raft 的不允许的！本小节只是用来举一种错误情况！

如 5.3 节中提到的那样，一旦当前任期内的某个日志条目以及存储到过半的服务器节点上，leader 就知道该日志可以被提交了。如果这个 leader 在提交某个日志条目之前崩溃了，以后的 leader 会尝试完成该日志条目的复制。然而，如果是之前任期内的某个日志条目已经存储到了过半的服务器节点上了，新任期内的 leader 也无法立即断定该日志条目已经被提交了。图 8 展示了一种情况：一个已经被存储到过半节点的老日志条目，仍然有可能会被未来的 leader 覆盖掉。

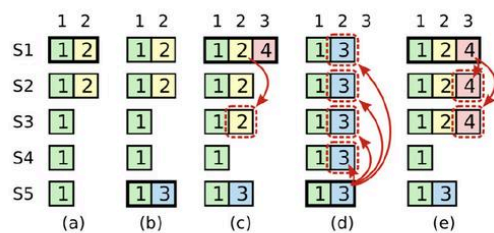


图8：展示了为什么 leader 无法判断处于老任期号内的日志条目是否已经被提交。

在 (a) 中，S1 是 leader，复制了索引位置 2 的日志条目给 S2，这时还没过半。

在 (b) 中，S1 宕机了，然后 S5 在任期 3 中通过 S3、S4 和它自己的投票赢得了选举，然后从客户端接收了一条不一样的日志条目放在了索引位置 2 上面。

在 (c) 中，S5 宕机了，S1 重启，此时 S1 和 S2 都可能成为 leader，假如 S1 赢得选举，然后 S1 继续复制它之前在任期 2 中放在索引 2 上的日志条目。此时，来自任期 2 的那条日志已经被复制到了集群中过半的节点上了，但是它还没被提交。

情况一：在 (d) 中，假如 S1 在提交日志之前宕机了，然后 S5 重启，这个时候，S5 最后的日志条目上的任期号比 S2、S3 和 S4 都大，所以它可以获得到 S2、S3、S4 和自己的投票成功当选 leader。S5 当选 leader 后，它就继续复制在任期 3 期间存储在索引位置 2 上的日志条目，那么该日志条目就会覆盖之前 S1 复制在节点 S1、S2、S3 索引 2 处的日志中。

情况二：在 (e) 中，如果在宕机之前，S1 在自己任期内复制了日志条目到大多数机器上。那么 S5 就不可能赢得选举，这种情况下，之前的所有日志也被提交了。

译者注：对图 8 的理解的补充。

参考：[知乎](#)

为了解决图 8 中描述的问题，Raft 永远不会通过计算副本数目的方式来提交之前任期内的日志条目。只有 leader 当期内的日志条目才通过计算副本数目的方式来提交。一旦当前任期内的某个日志条目以这种方式被提交（如图 8 中的 e），那么由于日志匹配特性（Log Matching），之前的所有日志条目也会被间接地提交。在某些情况下，leader 可以安全地断定一个老的日志条目已经被提交（例如，如果该条目已经被存储到每一个节点上了）。但是 Raft 为了简化问题，采取了上述描述的更加保守的方法。

Raft 会在提交规则上增加额外的复杂性，是因为当 leader 复制之前任期内的日志条目时，这些日志条目都保留原来的任期号。在其他的共识算法中，如果一个新的 leader 要重新复制之前任期里的日志时，它必须使用当前新的任期号。Raft 的做法使得更加容易推导出日志条目，因为它们自始至终都使用同一个任期号。另外，和其他的算法相比，Raft 中的新 leader 只需要发送更少的日志条目（其他算法中必须在它们被提交之前发送更多的冗余日志条目来给它们重新编号）。

5.4.3 安全性论证（Safety argument）

给出了完整的 Raft 算法后，我们现在可以更严格地来论证 leader 完整性特性（Leader Completeness Property）（这一讨论基于 9.2 节的安全性证明）。我们先假设 Leader Completeness Property 是不满足的，然后再推出矛盾来。

假设：

假设任期 T 的 leaderT 在任期内提交了一个日志条目，但是该日志条目没有存在未来某些任期的 leader 中，假设 U 是大于 T 的没有存储该日志条目的最小任期号，处在任期 U 的 leader 称为 leaderU。

论证：

1. 因为 leader 从来不删除或重写自己的日志条目，所以如果一个已提交的日志要做到不存在未来的 leaderU 中的话，那么它只可能在 leaderU 选举的过程中被丢失。
2. leaderT 将该日志复制给了集群中过半的节点，leaderU 从集群中过半的节点得到了投票。因此，至少有一个节点（这里称它为 voter）同时接收了来自 leaderT 的日志条目并且给 leaderU 投票了。

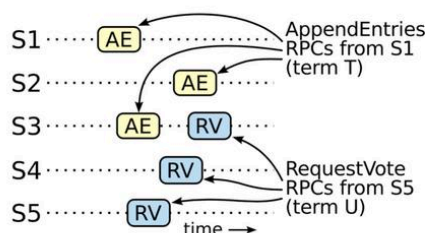


图9：如果任期 T 中的 leader（即 S1）在它任期内提交了一个新的日志条目，然后 S5 在后来的任期 U 中被选举为 leader，那么必然至少有一个节点（S3）不仅接收了来自 S1 的日志条目，而且给 S5 投票了。

1. voter 必然在给 leaderU 投票之前就已经接收了这个已经提交的日志条目了。否则，它就会拒绝来自 leaderT 的 AppendEntries RPC 请求，因为如果它在给 leaderU 投票之后再接收条目的话，那么它的当前任期号会比 T 大。

译者注：因为要举行 Leader election 的话需要开一轮新的任期，这个时候前一轮任期已经结束了。我们这里假设了 $T < U$ ，上述所说的已提交日志条目是在任期 T 中的，如果 voter 先投票的话，那么就说明它已经进入了任期 U 了，而 $U > T$ ，voter 是不可能接受 leaderT 的 AppendEntries 请求的。

2. 而且，voter 在给 leaderU 投票的时候，它依旧保有该日志条目，因为任何 U、T 之间的 leader 都包含该日志条目（因为我们前面假设了 U 是大于 T 的没有存储该日志条目的最小任期号），而且 leader 从来不会删除条目，并且 follower 只有再跟 leader 冲突的时候才会删除条目。
3. 该投票者把自己的选票投给 leaderU 的时候，leaderU 的日志至少跟 voter 一样新（可以更新），这就导致了以下的两个矛盾之一了。

4. 第一个矛盾：

如果 voter 和 leaderU 最后一个日志条目的任期号相同的话，那么 leaderU 的日志至少和 voter 的一样长，所以 leaderU 的日志一定包含 voter 日志中的所有日志条目。这是一个矛盾，因为 voter 包含了该已提交的日志条目，所以 leaderU 必定也包含该日志条目，而前面我们假设了 leaderU 是不包含的，这就产生了矛盾。

5. 第二个矛盾：

如果不是上面描述的情况的话，那么 leaderU 最后一个日志条目的任期号必然需要比 voter 的更大。此外，它还比 T 要大，因为 voter 拥有在任期号为 T 提交的日志条目，所以 voter 最后一个日志条目的任期号至少为 T。创建了 leaderU 的最后一个日志条目的之前的 leader 一定已经包含了该已被提交的日志条目（因为我们上面假设了 leaderU 是第一个没有该日志条目的 leader）。所以，根据日志匹配特性，leaderU 一定也包含了该已被提交的日志条目，这样也产生了矛盾。

6. 上述讨论就证明了假设是不成立的。因此，所有比 T 大的任期的 leader 一定包含了任期 T 中提交的所有日志条目。
7. 日志匹配特性保证了未来的 leader 也会包含被间接提交的日志条目，如图 8 (d) 中的索引 2。

通过 leader 的完整性特性，我们就可以证明图 3 中的状态机安全特性了，即如果某个节点已经将某个给定的索引处的日志条目应用到自己的状态机里了，那么其他的节点就不会在相同的索引处应用一个不同的日志条目。在一个节点应用一个日志条目到自己的状态机中时，它的日志和 leader 的日志从开始到该日志条目都是相同的，并且该日志条目必须被提交。现在考虑一个最小的任期号，在该任期中任意节点应用了一个给定的最小索引上面的日志条目，那么 Log 的完整性特性就会保证该任期之后的所有 leader 将存储相同的日志条目，因此在后面的任期中应用该索引上的日志条目的节点会应用相同的值。所以，状态机安全特性是可以得到保证的。

最后，因为 Raft 要求服务器节点按照日志索引顺序应用日志条目，再加上状态机安全特性，这样就意味着我们可以保证所有的服务器都会按照相同的顺序应用相同的日志条目到自己的状态机中了。

5.5 follower 和 candidate 崩溃 (Follower and candidate crashes)

到目前为止，我们只关注了 leader 崩溃的情况。follower 和 candidate 崩溃后的处理方式要比 leader 崩溃简单得多，而且它们的处理方式是相同的。如果一个 follower 或者 candidate 崩溃的话，后面发送给它们的 RequestVote 和 AppendEntries RPCs 都会失败。Raft 通过无限重试来处理这种失败。如果崩溃的节点重启了，那么这些 RPC 就会被成功地完成。如果一个节点在完成了一个 RPC，但是还没来得及响应就崩溃了的话，那么在它重启之后它会再次收到同样的请求。Raft 的 RPCs 都是幂等的，所以重复发送相同的 RPCs 不会对系统造成危害。实际情况下，一个 follower 如果接收了一个 AppendEntries 请求，但是这个请求里面的这些日志条目在它日志中已经有了，它就会直接忽略这个新的请求中的这些日志条目。

译者注：幂等

在编程中一个幂等操作的特点是其任意多次执行所产生的影响均与一次执行的影响相同。幂等函数，或幂等方法，是指可以使用相同参数重复执行，并能获得相同结果的函数。这些函数不会影响系统状态，也不用担心重复执行会对系统造成改变。例如，“setTrue()”函数就是一个幂等函数，无论多次执行，其结果都是一样的。更复杂的操作幂等保证是利用唯一交易号(流水号)实现。

5.6 时序和可用性 (Timing and availability)

Raft 中有一个要求就是 Raft 的安全性不能依赖于时序 (timing)：整个系统不能因为某些事件运行得比预期快一点或者慢一点就产生错误的结果。然而，可用性（即系统能够及时响应客户端的请求）不可避免的要依赖于时序。比如说，如果信息交换的时间比一般服务器崩溃所持续的时间还要长的话，那么 candidate 可能等不到赢得选举了，而缺少了一个稳定的 leader，Raft 将无法工作。

Raft 中时序最关键的地方就是 Leader election。只要整个系统满足下面的时间要求，Raft 就可以选举并维持一个稳定的 leader：

* 广播时间 (broadcastTime) << 选举超时时间 (electionTimeout) << 平均故障间隔时间 (MTBF) *

在这个不等式中，广播时间指的是一个节点并行地发送 RPCs 给集群中其他所有的节点并得到响应的平均时间。选举超时时间就是在 5.2 节中介绍的选举超时时间。平均故障间隔时间就是对于一台服务器而言，两次故障间隔时间的平均值。广播时间必须比选举超时时间小一个量级，这样 leader 才能够有效发送心跳信息来组织 follower 进入选举状态。再加上随机化选举超时时间的方法，这个不等式也使得无果选票（split vote）变得几乎不可能。而选举超时时间需要比平均故障间隔时间小上几个数量级，这样整个系统才可以稳定地运行。有了这个限制后，当 leader 崩溃后，整个系统**会有一段大约选举超时时间的时长不可用**，我们希望该情况在整个系统运行时间里只占一小部分。

广播时间和平均故障间隔时间是由系统决定的，但是选举超时时间是可以自定义的。Raft 的 RPCs 需要接收方将信息持久化地保存到稳定存储中，所以广播时间大约是 0.5ms ~ 20ms 之间，取决于存储的技术。因此，选举超时时间可能需要在 10ms ~ 500ms 之间。而大多数的服务器的平均故障间隔时间都在几个月甚至更长，所以很容易满足时间的要求。

6. 集群成员变更（Cluster membership changes）

到目前为止，我们都假设集群的配置（参与共识算法的服务器节点集合）是固定不变的。但是在实际情况中，我们有时候是需要去改变集群配置的，比如说在服务器崩溃的时候去更换服务器或者是更改副本的数量。尽管可以通过下线整个集群，更新所有配置，然后重启整个集群的方式来实现这个需求，但是这会导致集群在更改过程中是不可用的。另外，如果这个过程中存在一些操作需要人工干预，那么就会有操作失误的风险。为了避免这些问题，我们决定将**配置变更自动化**并将其纳入到 Raft 的共识算法中来。

为了使配置变更机制足够安全，在配置变更过程中不能存在任何一个时刻使得同一任期中选出两个 leader。遗憾的是，任何服务器直接从旧的配置转换为新的配置的方案都是不安全的。一次性自动地转换所有服务器的配置的不可能的，所以在转换期间整个集群可能划分为两个独立的大多数（如图 10 所示）。

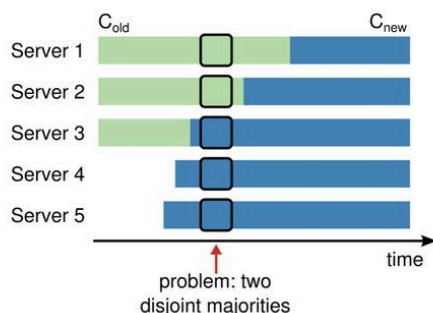


图10：直接从一种配置转到另一种配置是不安全的，因为每个节点会在不同的时间段进行转换。在图中的例子中，集群节点从 3 个变成 5 个。很不巧，有一个时间点是可能会选出两位 leader 的。一个获得过半处于旧配置的节点的支持，一个获得过半处于新配置的节点的支持。

译者注：图 10 补充

上图中，在中间位置 Server1 可以通过自身和 Server2 的选票成为 leader（满足旧配置下收到大多数选票的原则）；Server3 可以通过自身和 Server4、Server5 的选票成为 leader（满足新配置线，即集群有 5 个节点的情况下的收到大多数选票的原则）；此时整个集群可能在同一任期中出现了两个 leader，这和 Raft 协议是违背的。

为了保证安全性，配置变更必须采取一种**两段式方法（two-phase approach）**。目前有很多种两段式的实现。例如，有些系统（如 [22]）在第一阶段停掉旧的配置，所以在这个阶段不能处理用户的请求，然后在第二阶段启用新的配置。在 Raft 中，集群先切换到一个过渡的配置，我们称之为**联合共识（joint consensus）**。一旦联合共识配置已经被提交了，系统就可以切换到新的配置上了。**联合共识配置是新旧配置的并集：**

- 日志条目被复制给集群中处于新、老配置的所有节点
- 新、旧配置的节点都可能成为 leader
- 达成一致（针对选举和提交）需要分别得到在两种配置上过半的支持

联合共识允许每一个节点在不妥协安全性的前提下，在不同的时刻进行配置转换过程。此外，联合共识还允许在集群配置变更期间响应客户端的请求。

集群配置在复制日志中以特殊的日志条目来存储和通信。图 11 展示了配置变更的过程。

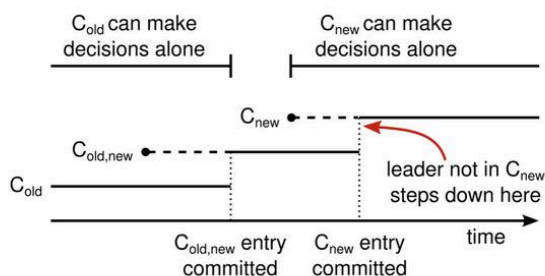


图11: 配置变更过程

虚线表示已创建但未提交的配置项，实线表示最新提交的配置项。leader 首先在日志中创建一个 $C_{old,new}$ 的配置条目，并将其交给集群中大多数处于旧配置和大多数处于新配置的节点。然后 leader 再创建一个 C_{new} 的日志条目，并将其提交给过半的处于新配置的节点。处于新配置的节点和处于旧配置的节点不可能同时独立地做出决定。

当 leader 接收到一个更新配置的请求的时候，它就创建一个联合共识日志条目 Cold,new，并以前面描述的方式复制该条目。一旦某个节点将该配置日志条目增加到自己的日志中。那么这个节点就会用该配置来做出未来的所有决策（一个节点总是使用日志中最新的配置，无论该日志是否已经被提交）。这就意味着 leader 会使用 Cold,new 的规则来判断 Cold,new 日志条目是什么时候被提交的。如果 leader 崩溃了，新的 leader 有可能处于 Cold 配置，也可能处于 Cold,new 配置，这取决于赢得选举的 candidate 是否已经接收到了 Cold,new 配置。在任何情况下，处于 Cnew 状态的节点在此期间都是不能单独做出决定的。

当 Cold,new 被提交了,那么 Cold 和 Cnew 都不能在没有得到对方认可的情况下做出决定,并且 **Leader 完整特性 (Leader Completeness Property)** 保证了只有拥有 Cold,new 日志的 candidate **有可能被选为 leader**。所以现在 leader 就可以安全地创建一个描述 Cnew 的日志条目并将其复制给集群中的其他节点了。一样的,新的配置被节点收到后就会立刻生效。当新的配置在 Cnew 的规则下被提交之后,旧配置就变得无关紧要了,处于旧配置的节点也可以关闭了。如图 11 所示,没有任何一个时刻 Cold 和 Cnew 是可以单独做决定的,这保证了安全性。

关于配置变更有三个问题需要解决：

- 第一个问题：新的节点可能在一开始并没有存储任何的日志条目。当这些节点以这种状态加入到集群中的时候，它们需要一段时间来更新自己的日志，以便赶上其他节点，在这个时间段里面它们是不可能提交一个新的日志条目的。**为了避免因此造成的系统短时间的不可用，Raft 在配置变更前引入了一个额外的阶段。在该阶段中，新的节点以没有投票权身份加入到集群中来（leader 会把日志复制给它们，但是考虑过半的时候不需要考虑它们）。**一旦新节点的日志已经赶上了集群中的其他节点，那么配置变更就可以按照之前描述的方式进行了。
- 第二个问题：leader 有可能不是新配置中的一员（译者注：也就是说这个 leader 后面是需要被下线的）。在这种情况下，leader 一旦提交了 Cnew 日志条目，它就会退位为 follower（译者注：Cold,new 状态下依旧可用）。这就意味着有这样一段时间（leader 提交 Cnew 期间）：leader 管理着一个不包括自己的集群，它会复制日志给其他节点，但是算副本数量的时候不会算上自己。leader 转换发生在 Cnew 被提交的时候，因为这是新配置可以独立运行的最早时刻（在这个时刻之后，一定是从 Cnew 中选出新的 leader）。在这个时间点之前，有可能只能从 Cold 中选出 leader。
- 第三个问题：那么被移除的节点（不处于 Cnew 状态的节点）有可能会扰乱集群。这些节点将不会收到心跳信息，所以当选举超时，它们就会进行新的选举过程。它们会发送带有新任期号的 RequestVote RPCs，这样会导致当前的 leader 回到 follower 状态，然后选出一个新的 leader。但是这些被移除的节点还是会收不到心跳，然后再次超时，再次循环这个过程，导致系统的可用性很差。
为了避免这个问题，当节点认为当前有 leader 存在时，节点会忽略 RequestVote RPCs。具体来说，当一个节点在最小选举超时时间内收到一个 RequestVote RPC，它不会更新它的任期或授予它的投票。这不会影响正常的选举，每个节点在开启一轮选举之前，它会至少等待一次最小选举超时时间。相反，这有利于避免被移除的节点的扰乱：如果一个 leader 能够发送心跳给集群，那它就不会被更大的任期号废黜。

7. 日志压缩

在正常情况下，Raft 的日志会随着客户端请求的增加而不断增长。但在实际系统中，日志不可能无限制地增长。随着日志越来越长，它会占用越来越多的空间，并且需要花更多的时间来重新执行日志中的日志条目。如果没有一定的机制来清除日志中积累的过期的信息，那么最终一定会影响系统的可用性。

快照技术 (snapshotting) 是日志压缩最简单的方法。在快照技术中，某个时间点下的前整个系统的状态都会以快照的形式持久化起来，然后该时间点之前的日志会被全部丢弃。快照技术被使用在 Chubby 和 ZooKeeper 当中，接下来的章节会介绍 Raft 中的快照技术。

增量压缩方法 (Incremental approach to compaction)，例如**日志清洗 (log cleaning)** [36] 和**日志结构合并树 (log-structured merge trees)** [30, 5]，都是可行的。这些方法每次只对一小部分数据进行操作，这样就分散了压缩的负载压力。首先，选择一个积累了大量被删除或被覆盖的对象的数据区域，然后重写该区域内还活着的对象，之后释放该区域。和快照技术相比，这需要大量额外的机制，并且增加了更多的复杂性，快照技术通过操作整个数据集来简化问题。虽然日志清理需要对 Raft 进行修改，但是状态机可以使用与快照技术相同的接口来实现 LSM（日志结构合并）树。

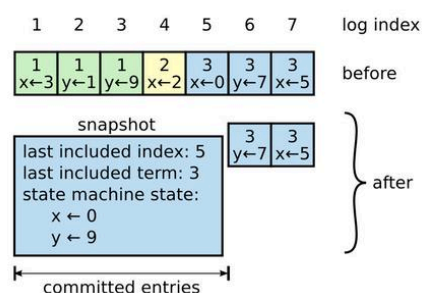


图12：一个节点用一个新的快照来覆盖它日志中已经提交的条目（索引1-5），该快照只存储了当前的状态（在这个例子中表示变量 x 和 y）。快照的 last included index 和 last included term 被保存用来定位日志中 index 6 之前的快照。

图 12 展示了 Raft 快照技术的基本思想。每一个节点独立地生成快照，快照中只包含自己日志中已经被提交的条目，这个过程主要的工作是状态机将自己的状态写入快照中。Raft 在快照中还保留了少量的元数据：

- last included index：指的是最后一个被快照取代的日志条目的索引值（状态机最后应用的日志条目）
- last included term：指的是该条目所处的任期号

保留这些元数据是为了支持快照后第一个条目的 AppendEntries 一致性检查，因为该条目需要一个之前的日志索引和任期号。为了支持集群成员变更（第 6 节中讨论的），快照中还包含日志中到 last included index 为止的最新的配置。一旦节点完成了快照的写入，它可能就会删除 last included index 及之前的所有日志条目，以及之前的快照。

尽管通常情况下，节点都是**独立生成快照**的，但是 leader 不可避免偶尔需要发送快照给一些落后的 follower。这通常发生在 leader 已经丢弃了需要发给 follower 的下一条日志条目的时候。幸运的是，这种情况在正常操作中是不会出现的：一个与 leader 保持同步的 follower 通常都会拥有该日志条目。**不过如果一个 follower 运行比较缓慢，或者是它刚加入集群，那么它就可能会没有该日志条目。**这个时候 leader 会通过网络将该快照发送给该 follower，以使得该 follower 可以更新到最新的状态。

InstallSnapshot RPC		InstallSnapshot RPC	
Invoked by leader to send chunks of a snapshot to a follower. Leaders always send chunks in order.		由 leader 调用，用来将快照块发送给 follower，leader 会按顺序发送快照块。	
Arguments:		Arguments:	
term	leader's term	term	leader 的任期
leaderId	so follower can redirect clients	leaderId	leader 的 ID，这样 follower 才能重定向客户端的请求
lastIncludedIndex	the snapshot replaces all entries up through and including this index	lastIncludedIndex	最后一个被快照取代的日志条目的索引值
lastIncludedTerm	term of lastIncludedIndex	lastIncludedTerm	lastIncludedIndex 所处的任期号
offset	byte offset where chunk is positioned in the snapshot file	offset	数据块在快照文件中位置的字节偏移量
data[]	raw bytes of the snapshot chunk, starting at offset	data[]	从偏移量 (offset) 开始快照块的原始字节数据
done	true if this is the last chunk	done	表示是否是最后一个快照块
Results:		Results:	
term	currentTerm, for leader to update itself	term	当前的任期，供 leader 自我更新
Receiver implementation:		Receiver implementation:	
1. Reply immediately if term < currentTerm 2. Create new snapshot file if first chunk (offset is 0) 3. Write data into snapshot file at given offset 4. Reply and wait for more data chunks if done is false 5. Save snapshot file, discard any existing or partial snapshot with a smaller index 6. If existing log entry has same index and term as snapshot's last included entry, retain log entries following it and reply 7. Discard the entire log 8. Reset state machine using snapshot contents (and load snapshot's cluster configuration)		1. 如果 term < currentTerm，立即回复 2. 如果是第一个 chunk（偏移量为 0），创建新的快照文件 3. 按给定偏移量将数据写入快照文件 4. 如果 done 为 false，则回复并等待更多数据块 5. 保存快照文件，丢弃索引较小的现有快照或部分快照 6. 如果现有日志条目与快照最后包含的条目具有相同的 index 和 term，则保留它后面的日志条目并进行回复 7. 丢弃整个日志 8. 使用快照内容重置状态机（并加载快照的集群配置）	

图13：InstallSnapshot RPC 的浓缩。快照被分割成块来用于传输，这给了 follower 每个快照块的生命信号，所以 follower 可以根据这个信号来重置它的选举计时器。

这个时候 leader 使用了一种新的 RPC 来发送快照给那些太落后的 followers，如图 13 所示，这种 RPC 叫做 **InstallSnapshot**。当一个 follower 通过这种 RPC 收到快照的时候，它必须决定如何处理当前已经存在的日志条目。通常情况下，这份快照会包含接受者日志没有的信息。**所以这种情况下 follower 会丢弃它的整个日志，它的日志会全部被快照取代，并且可能有与快照冲突的未提交的条目。**相反，如果一个 follower 收到一个描述其日志前缀的快照（可能是由于重传或错误），则被快照覆盖的日志条目将被删除，但是快照之后的条目仍然有效，且必须要保留。

这种快照的方式违反了 Raft 的 strong leader 原则，因为 follower 可能在不知道 leader 的情况下创建快照。但是我们认为这种违背是合乎情理的。leader 的存在，是为了防止在达成共识的时候产生冲突，但是在**创建快照的时候，共识已经达成了**，因此没有决策会出现冲突。这种情况下，数据还是跟之前一样，只能从 leader 流向 follower，只不过现在允许 follower 可以重新组织它们的数组而已。

我们曾经考虑过一种可替代的方案，那就是只有 leader 可以创建快照，然后由 leader 将这份快照发送给其他所有的 follower。但是，这种方案有两个缺点：

1. 发送快照给每个 follower 会**浪费网络带宽和延缓了快照处理过程**。实际上每一个 follower 已经拥有了创建自己快照所需要的全部信息了，所以很显然，follower 根据本地的状态创建快照要比通过网络来接收别人发过来的要更加实惠。
2. 这会造成 leader 的实现更加复杂。比如说，leader 发送快照给 follower 的同时要能够做到**并行地将新的日志条目发送给它们**，这样才不会阻塞新的客户端请求，这就复杂得多了。

还有两个问题会影响快照的性能：

1. 每一个节点必须判断何时去生成快照。如果一个节点生成快照的频率太高，那么就会浪费大量的磁盘带宽和其他资源；如果一个节点生成快照的频率太低，那么就要承担耗尽存储

2. 容量的风险，同时也增加了重启时重新执行日志的时间。

一个简单的策略就是当日志大小达到一个固定的阈值的时候就生成一份快照。如果这个阈值设置得显著大于期望的快照的大小，那么快照的磁盘带宽开销将较小。

3. 第二个影响性能的就是写快照需要花费一定的时间，而我們又不希望它会影响到正常的操作。

解决方案就是使用 **写时复制的技术 (copy-on-write)**，这样新的更新就可以在不影响正在写的快照的情况下被接收。例如，具有泛型函数结构的状态机天然支持这样的功能。另外，操作系统对写时复制技术的支持（如 Linux 上的 fork）可以被用来创建整个状态机的内存快照（我们的实现用的就是这种方法）。

8. 客户端交互

本节介绍客户端如何和 Raft 进行交互，包括客户端如何找到 leader 和 Raft 是如何支持线性化语义的 [10]。这些问题对于所有的基于共识算法的系统都是存在的，Raft 的解决方案也跟其他的系统差不多。

Raft 的客户端们将所有的请求发送给 leader。当客户端第一次启动的时候，它会随机挑选一个节点来进行通信。如果客户端首选的不是 leader，那么被客户端选中的节点就会拒绝客户端的请求并且提供关于它最近收到的 leader 的信息（AppendEntries RPC 包含了 leader 的网络地址）。如果 leader 崩溃了，客户端请求就会超时，这个时候客户端需要随机选择一个节点来重试发送请求。

我们对 Raft 的期许是希望它可以实现线性化语义（即每次操作看起来似乎都是在调用和响应之间的某个点上即时执行一次）。但是，按照上面描述的，Raft 可能会对同一条指令执行多次。例如，如果 leader 在提交了某个日志条目后，在还没来得及响应客户端的时候就崩溃了，那么客户端会和新的 leader 重试该指令，这就造成了同一指令被执行了两次。解决方案是客户端对于每一条指令都赋予一个唯一的序列号。然后，状态机跟踪每个客户端已经处理的最新的序列号以及相关联的响应。如果状态机接收到了一条已经执行过的指令了，就立即作出响应，并且不会重复执行该指令。

只读操作（Read-Only）可以直接处理而不记录日志。但是，如果不采取任何措施的话，这可能会有返回过期数据（stale data）的风险。**因为 leader 响应客户端请求的时候它可能已经被新的 leader 代替了，但是它还不知道自己已经不是最新的 leader 了。**

译者补充：**为什么一个 leader 好好的会有另外一个 leader 出现？**

参考：<https://segmentfault.com/a/1190000039264427>

实际上，老的 leader 可能不会马上消失，例如：网络分区将 leader 与集群的其余部分分隔，其余部分选举出了一个新的 leader。然后老的 leader 崩溃后重新连接，可能会不知道新的 leader 已经被选出来了。

线性化的操作肯定不会返回过期的数据。Raft 需要使用两个额外的预防措施来在不适用日志的时候保证这一点。

1. leader 必须拥有那些已提交的日志条目的最新信息。Leader 完整性特性 (Leader Completeness Property) 保证了 leader 一定拥有所有已被提交的日志条目，但是在它任期刚开始的时候，它可能还不知道哪些是已经被提交的。为了知道这些信息，它需要在它的任期里提交一个日志条目。

Raft 通过让 leader 在任期开始的时候提交一个空的日志条目到日志中来解决该问题。（译者注：这就是前面 5.4.2 节提到的 no-op 日志）

2. leader 在处理只读请求的时候必须检查自己是否已经被替代了（因为如果一个新 leader 被选出来了，那么这个旧 leader 的数据可能就过时了）。
Raft 通过让 leader 在响应只读请求之前，先和集群中过半的节点交换一次心跳信息来解决该问题。另一种可选的方案，leader 可以依赖心跳机制来实现一种租约的形式 [9]，但是这种方式的安全性需要依赖于时序（假设时间误差是有界的）。

9. 算法实现与评估

我们已经实现了 Raft 作为复制状态机的一部分，该状态机存储了 RAMCloud [33] 的配置信息，并帮助 RAMCloud 协调器进行故障转移。这个 Raft 实现大概包含了 2000+ 行 C++ 代码，但是这里面没有包含测试、注释和空行。这些代码是开源的 [23]。同时也有大约 25 个其他独立的第三方、针对不同的开发场景、基于这篇论文草稿的开源实现。同时，很多公司已经部署了基于 Raft 算法的系统了。

本节剩下的篇幅将从三个方面来评估 Raft 算法：

- 可理解性
- 正确性
- 性能

9.1 可理解性

为了衡量 Raft 相对于 Paxos 的可理解性，我们针对高层次的本科生和研究生，在斯坦福大学的高级操作系统课程和加州大学伯克利分校的分布式计算课程上，进行了一项实验研究。我们为 Raft 和 Paxos 分别录制了一个视频教程，并且准备了相应的小测验。其中 Raft 课程覆盖了本篇论文除了日志压缩之外的全部内容，而 Paxos 课程涵盖了创建一个与 Raft 等价的复制状态机的全部资料，包括 single-decree Paxos、multi-decree Paxos、重新配置和一切实际系统需要的性能优化（比如 leader 选举）。这个小测验主要是测试一些对算法的理解和解释一些边缘情况。每个学生都是看完第一个视频，然后做对应的测验，然后再看第二个视频，再做第二份测验。为了解释个人表现与从第一部分研究中获得的经验差异的原因，大约有一半的学生先进行 Paxos 的部分，然后另一半学生先进行 Raft 的部分。我们通过计算参与人员的每一份测验的得分来看参与者是否更加容易理解 Raft 算法。

我们尽可能的使得在比较 Raft 和 Paxos 过程中是公平的。这个实验从两个方面偏向了 Paxos：

1. 43 个参与者中有 15 个人在之前有一些 Paxos 的经验
2. Paxos 视频教程的时长要长 14%

如表格 1 总结的那样，我们采取了一些措施来减轻这种潜在的偏向。我们所有的材料都可供审查 [28, 31]。

关注点	缓和偏向采取的手段	可供查看的材料
相同的讲课质量	两份教程采用同一个讲师。Paxos 的教程是在现有的一些大学使用的材料基础上改进的。Paxos 的教程要长 14%。	视频
相同的测验难度	问题以难度分组，在两个测验里成对出现。	小测验
公平评分	使用 评价量规 。随机顺序打分，两个测验交替进行。	评分细则

表格1：考虑到潜在的实验偏向，我们对于每种情况的解决方法，以及相应的材料。

平均上看，参与者在 Raft 测验上的得分要比在 Paxos 测验上的得分高处 4.9 分（在 60 分中，Raft 的平均得分是 25.7 分，Paxos 的平均得分是 20.8 分）。图 14 展示了每个参与者的得分。配对 t 检验（paired t-test）表明，在 95% 的[置信度](#)下，Raft 分数的真实分布的平均值至少要比 Paxos 的大 2.5 分。

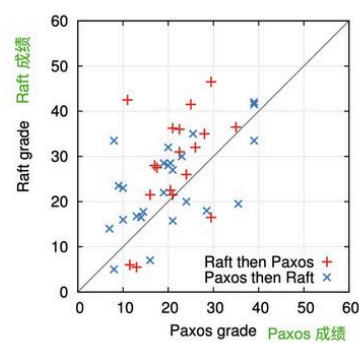


图14：一个散点图表示了 43 个学生在 Paxos 和 Raft 测验中的成绩。在对角线之上的点表示在 Raft 中获得了较高分数的学生。

我们也建立了一个[线性回归模型](#)来预测一个新的学生的测验成绩，这个模型基于以下三点：

1. 他们使用的是哪个测验
2. 之前对于 Paxos 的经验
3. 学习算法的顺序

该模型预测，对小测验的选择会产生 12.5 分的有利于 Raft 的差别，这很明显高于观察到的 4.9 分的分差。这是因为实际上许多的学生之前有学习过 Paxos，这对 Paxos 的有很大帮助的，但是对 Raft 的帮助就较小了。但是奇怪的是，模型预测对于先进行 Paxos 小测验的人而言，Raft 的得分低了 6.3 分。虽然我们不知道这是为什么，但是这似乎在统计上是有意义的。

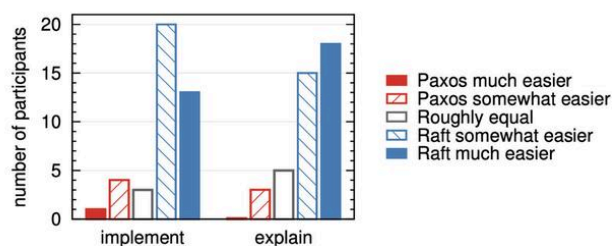


图15：使用 5 分制，参与者被询问他们认为哪种算法在实现一个可用、正确和高效的系统中更容易实现，以及哪种算法更容易向计算机科学研究生解释。

我们同时也在测验之后对参与者进行了调查，调查的内容是他们认为哪个算法更容易去实现或解释。这些调查结果展示在图 15。调查结果是碾压性的，结果表明 Raft 算法更加容易实现和解释（41 人中的 33 个）。然而，这种自我报告的感觉可能没有参与者的测试分数来得可靠，而且参与者可能由于我们假设 Raft 更容易理解而存在偏向。

在参考文献 [33] 中有一个关于 Raft 用户学习的更加详细的讨论。

9.2 正确性

在第 5 节中，我们已经对[共识机制](#)制定了正式的规范并且对其安全性做了证明。这份正式的规范使用 TLA+ 规范语言 [17] 使图 2 中对算法的总结的信息非常清晰。它差不多有 400 行并且作为了我们要证明的核心。同时这份规范对于任何想实现 Raft 的人都是十分有用的。我们用 TLA 证明系统 [7] 机械地证明了日志完整性 (Log Completeness Property)。然而，这个证明依赖的约束前提还没有被机械证明（例如，我们还没有证明规范中的[类型安全](#)）。而且，我们已经编写了状态机安全特性的非正式证明 [31]，它是完整的（它仅依赖于规范）和相对精确的（大约 3500 字长）。

9.3 性能

Raft 的性能跟其他像 Paxos 的共识算法很接近。在性能方面，最重要的关注点就是，当一个 leader 被选举出来后，它要在什么时候复制新的日志条目。Raft 通过很少量的消息包（一轮从 leader 到集群中过半节点的的消息传递）就解决了这个问题。同时，进一步提升 Raft 的性能也是有可能的。比如说，很容易通过支持批量操作和[管道操作](#)来提高吞吐量和降低延迟。对于其他共识算法已经提出过很多性能优化方案，其中很多都可以应用到 Raft 上，但是我们暂时把这些工作放到未来的工作中。

我们使用我们自己的 Raft 实现来衡量 Raft 的 leader election 算法的性能并且回答两个问题：

1. leader 选举过程收敛是否足够快？
2. 在 leader 崩溃之后，最小的系统崩溃时间是多久？

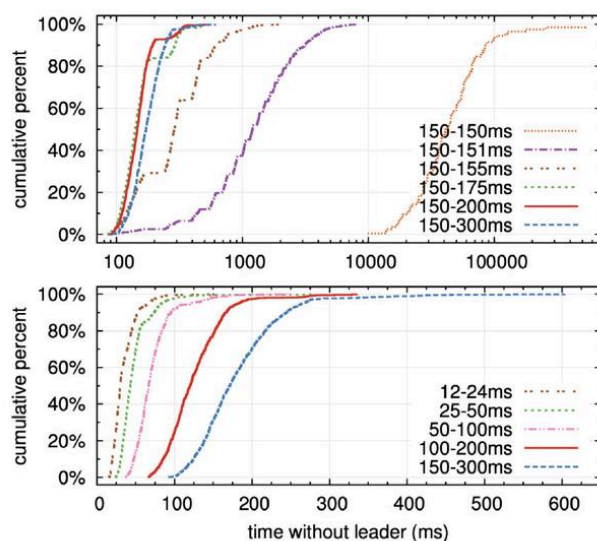


图16：检测并替换一个崩溃的 leader 所需要的时间。

上面的图观察了在选举超时时间上的随机化程度，下面的图考察了最小的选举超时时间。每一条线代表了 1000 次实验（除了 150-155 毫秒只试了 100 次）以及相应的确定的选举超时时间。比如说，“150-155ms”的意思是，选举超时时间从 150~155ms 这个区间内随机选择并确定下来。这个实验在一个拥有 5 个节点的集群上进行，其广播时延大约是 15ms。对于 9 个节点的集群，结果也差不多。

为了衡量 leader election 的性能，我们反复使一个拥有 5 个节点的集群的 leader 宕机，并计算它检测崩溃和重新选一个新的 leader 所需的时间（见图 16）。为了构建一个最坏的情景，我们使各个节点中的日志长度都是不同的，这样某些 candidate 是无法成为 leader 的。而已，为了尽可能出现无结果的投票（split vote）情况，我们的测试脚本在终止 leader 的进程之前从 leader 那触发了一次同步的发送了一次心跳广播（类似于 leader 在崩溃前复制一个日志条目给其他节点）。leader 在其心跳间隔内均匀随机地崩溃，这个心跳间隔也是所有测试中最小选举超时时长的一半。因此，**最小宕机时间大约就是最小选举超时间的一半**。

图 16 中上面的图表明，只需要在选举超时时间上使用很小的随机化就可以大大避免出现没有结果的投票的情况。在没有随机化的情况下（译者注：见图 16 中上面的图右边的橙色虚线），由于出现了很多没有结果的投票的情况，leader election 往往都需要花费超过 10s 的时间。仅仅加入 5ms 的随机化时间，就大大改善了选举过程，现在平均的宕机时间只有 287ms。继续增大随机性可以大大改善最坏的情况：通过增加 50ms 的随机化时间，最坏的完成情况（即完成 1000 次实验）只需要 513 ms。

图 16 中下面的图表明，通过减少选举超时时间可以减少系统的宕机时间。在选举超时时间为 12~24ms 的情况下，只需要平均 35ms 就可以选举出新的 leader（最长的一次花费了 152ms）。然而，进一步降低选举超时时间可能就会违反 Raft 不等式的要求。

广播时间 (broadcastTime) << 选举超时时间 (electionTimeout) << 平均故障间隔时间 (MTBF)

因为这会使得在其他节点开启新一轮的选举之前，当前的 leader 要完成发送一次心跳广播变得很难。这会造成不必要的 leader 更换，从而降低了系统的可用性。我们推荐使用一个更为保守的选举超时时间，比如 150~300ms。这样的时间不大可能导致不必要的 leader 更换，同时还能提供不错的可用性。

10. 相关工作

现在已经有很多关于共识算法相关的产物了，其中很多都属于以下类别之一：

- Lamport 对于 Paxos 的最初的描述 [15]，以及尝试将 Paxos 解释地更清晰的描述 [16, 20, 21]。
- 关于 Paxos 的更详尽的描述，补充遗漏的细节并修改算法，使得可以提供更加容易的实现基础 [26, 39, 13]。
- 实现共识算法的系统，例如 Chubby [2, 4]，ZooKeeper [11, 12] 和 Spanner [6]。对于 Chubby 和 Spanner 的算法并没有公开发表其技术细节，尽管他们都声称是基于 Paxos 的。ZooKeeper 的算法细节已经发表，但是和 Paxos 着实有着很大的差别。
- 对于 Paxos 的性能优化 [18, 19, 3, 25, 1, 27]。
- Oki 和 Liskov 的 Viewstamped Replication (VR)，一种和 Paxos 差不多的替代算法。原始的算法描述 [29] 和分布式传输协议耦合在了一起，但是核心的共识算法在最近更新的版本 [22] 里被分离了出来。VR 使用了一种基于 leader 的方法，和 Raft 有很多相似之处。

Raft 和 Paxos 最大的不同就在于 Raft 的**强领导性 (strong leadership)**。Raft 将 leader election 作为共识协议中非常重要的一环，并且将尽可能多的功能集中到了 leader 身上。这种方法使得算法更加简单和更容易理解。比如说，在 Paxos 中，leader election 和基本的共识协议是正交的：它只是作为一种性能优化，而不是实现共识所必需的。然而，这带来了很多额外的机制：

- Paxos 中包含了一个两段式的基本共识协议
- Paxos 中还包含了一个单独的 leader election 机制

相比之下，Raft 将 leader election 直接纳入了共识算法并且将其作为共识两阶段中的第一个阶段，这使得 Raft 使用的机制要比 Paxos 少得多。

像 Raft 一样，VR 和 ZooKeeper 也是基于 leader 的，因此他们也拥有一些 Raft 的优点。但是，Raft 比 VR 和 ZooKeeper 拥有更少的机制。因为 Raft 尽可能的减少了非 leader 者的功能。例如，Raft 中日志条目都遵循着从 leader 发送给 follower 这一个方向：AppendEntries RPCs 是向外发送的。在 VR 中，日志条目的流动是双向的（leader 可以在选举过程中接收日志）；这就导致了额外的机制和复杂性。根据 ZooKeeper 公开的资料看，它的日志条目也是双向传输的，但是它的实现更像 Raft。

跟我们上述提到的其他基于共识性的日志复制算法相比，Raft 的消息类型更少。例如，我们计算了一下 VR 和 ZooKeeper 用来实现基本功能和集群成员变更（不包括日志压缩和客户端交互，因为这些都比较独立且和算法关系不大）所需要的消息类型。VR 和 ZooKeeper 都分别定义了 10 种不同的消息类型。相比之下，**Raft 只有 4 种消息类型（两种 RPC Request 及其对应的两种 RPC Response）**。Raft 的消息的消息量比其他算法的要大一点，但总的来说，它们更加简单。另外，VR 和 ZooKeeper 都在 leader 改变的时候传输了整个日志，所以这些算法为了能在实践中使用，就不得不增加额外的消息类型了。

Raft 的强 leader 模型简化了整个算法，但是同时也排斥了一些性能优化的方法。例如，平等主义 Paxos (EPaxos) 在某些没有 leader 的情况下可以达到很高的性能 [27]。平等主义 Paxos 充分发挥了在状态机指令中的交换性。任何服务器都可以在一轮通信下就提交指令，除非其他指令同时被提出了。然而，如果指令都是并发的被提出，并且互相之间不通信沟通，那么 EPaxos 就需要额外的一轮通信。因为任何服务器都可以提交指令，所以 EPaxos 在服务器之间的负载均衡做的很好，并且很容易在 WAN 网络环境下获得很低的延迟。但是，他在 Paxos 上增加了非常明显的复杂性。

一些集群成员变更的方法已经被提出或者在其他的工作中被实现，包括 Lamport 的原始的讨论 [15]，VR [22] 和 SMART [24]。我们选择使用联合共识的方法是因为它利用了共识协议的其余部分，这样我们只需要很少的一些机制就可以实现成员变更。Lamport 的基于 α 的方法之所以没有被 Raft 选择是因为它假设在没有 leader 的情况下也可以达到共识性。和 VR 和 SMART 相比较，Raft 的重新配置算法可以在不限制正常请求处理的情况下进行。相比之下，VR 在配置变更期间需要停止所有正常的处理过程，而 SMART 对未完成请求的数量实施了类似 α 方法的限制。另外，和 VR、SMART 相比，Raft 的方法也只需要增加更少的额外机制来实现。

11. 结论

算法的设计通常以**正确性、效率和简洁性**为主要目标。虽然这些都是有价值的目标，但我们相信**可理解性**同样重要。在开发人员将算法转化为实际实现之前，其他任何目标都不能实现，而实际实现将不可避免地偏离和扩展发布的形式。除非开发人员对算法有深刻的理解，并能对算法有直观的认识，否则他们很难在实现中保留算法理想的特性。

在本文中，我们讨论了分布式共识的问题，在这个问题上，一个被广泛接受但难以理解的算法：Paxos，多年来一直让学生和开发人员非常挣扎。我们开发了一种新的算法：Raft，我们已经证明它比 Paxos 更容易理解。我们也相信 Raft 会为系统建设提供更好的基础。将可理解性作为主要设计目标改变了我们处理 Raft 设计的方式。随着设计的进展，我们发现自己反复使用了一些技术，比如分解问题和简化状态空间。这些技术不仅提高了 Raft 的可理解性，而且使我们更容易证实它的正确性。